IOT-ENABLED LITHIUM-ION BATTERY MANAGEMENT SYSTEM USING ESP32

Shriyanshu Naik[1]
Department of Computer Science Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, Maharashtra, INDIA
naikshriyanshu21@gmail.com

Hritvi Bhole[1]
Department of Computer Science Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, Maharashtra, INDIA
hritvi1006@gmail.com

Fariha Naikwadi[1]
Department of Computer Science Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, Maharashtra, INDIA
farihanaikwadi@gmail.com

Mandar Deshmukh[1]
Department of Computer Science Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, Maharashtra, INDIA
manndarrr@gmail.com

Dr. Vitthal Gutte[2]
Department of Computer Science Engineering and Technology
Dr. Vishwanath Karad MIT World Peace University
Pune, Maharashtra, INDIA
vitthalgutte2014@gmail.com

***Abstract*—The proliferation of battery-powered systems, especially in Electric Vehicles (EVs) and portable electronics, necessitates intelligent and reliable Battery Management Systems (BMS). Traditional BMS solutions often lack remote monitoring capabilities, are costly, and have limited scalability. This paper presents the design and implementation of a low-cost, smart BMS leveraging the ESP32 microcontroller. The proposed system provides real-time monitoring of critical battery parameters, including voltage, temperature, and State of Charge (SOC), through a cloud-based dashboard. The system integrates a voltage divider for voltage sensing and a thermistor for temperature measurement. A relay module controlled by the ESP32 manages the charging process, incorporating over-voltage and over-temperature protection to enhance battery safety and longevity. By utilizing the ESP32's built-in Wi-Fi capabilities, the system transmits data to the Arduino IoT Cloud, enabling remote access and data visualization. This research demonstrates a scalable and cost-effective solution that significantly improves upon conventional battery monitoring by adding IoT connectivity and intelligent control.**

Keywords—Battery Management System (BMS), ESP32, Internet of Things (IoT), State of Charge (SOC), Lithium-Ion, Remote Monitoring.

I. INTRODUCTION

The global push towards electrification and sustainable energy has placed rechargeable batteries, particularly Lithium-ion (Li-ion), at the core of modern technology [1]. Their high energy density, favorable power-to-weight ratio, and long cycle life have made them the dominant energy storage solution in a vast range of applications, from handheld consumer electronics to utility-scale grid storage systems. A Battery Management System (BMS) is an essential electronic system that manages a rechargeable battery pack. Its primary responsibility is to ensure the battery operates within its **Safe Operating Area (SOA)**. This involves protecting the battery from conditions like over-voltage, under-voltage, over-current, and extreme temperatures, all of which can lead to irreversible capacity loss, internal damage, or even hazardous events like thermal runaway [2].

However, many existing BMS solutions, especially those in non-critical applications, are localized systems. They perform their protective functions adequately but lack the ability for remote monitoring, data logging, and intelligent diagnostics. Commercial-grade systems, such as those found in modern EVs are highly sophisticated but also proprietary, complex, and prohibitively expensive for smaller-scale projects or for retrofitting into existing systems [7]. This creates a significant technological gap: there is a growing need for a BMS that is not only safe, reliable but also intelligent, affordable, and connected within the modern IoT ecosystem.

This paper addresses this gap by proposing and documenting a Smart Battery Management System built around the versatile and cost-effective ESP32 microcontroller. While other ESP32-based BMS projects exist, their methods for estimating State of Charge (SOC) often rely on simplistic linear voltage-to-percentage mapping. This method is known to be highly inaccurate for Li-ion batteries, which have a non-linear voltage curve.

The primary contribution of this work is the design and implementation of a computationally-efficient piecewise linear interpolation algorithm for SOC estimation.

By storing a lookup table of known voltage-to-SOC data points from the battery's OCV curve in the ESP32's memory, our system achieves a significantly more accurate and reliable SOC reading than other low-cost designs. This is accomplished without requiring the high computational overhead of advanced methods like Kalman filters. This

979-8-3315-9442-8/25/$31.00 ©2025 IEEE

allows any user—from a hobbyist to a small-scale system integrator—to not only monitor their batteries remotely but to do so with a higher degree of accuracy, enhancing both safety and usability.

II. RELATED WORK

The field of battery management has seen significant research, with a growing focus on integrating IoT for enhanced functionality and data-driven insights. Several studies have explored different facets of this integration, forming the foundation upon which this work is built.

For instance, the work presented by Dr. R. Usha, et al. [1], details an IoT-based battery monitoring system for EVs that also utilizes the ESP32 microcontroller. Their system successfully demonstrates the viability of this approach for real-time tracking by collecting key parameters and transmitting them to the Arduino IoT cloud for visualization. Similarly, another study focuses on a generalized BMS design that emphasizes fault tolerance and modularity through the concept of Smart Battery Modules (SBMs), although this approach results in a more complex and costly architecture. The work by Deshpande et al. also highlights a comparable prototype BMS using the older ESP8266 processor to display battery parameters on the BLYNK IoT platform, further validating the use of Wi-Fi-enabled microcontrollers in this domain [9].

The challenge of accurately estimating the State of Charge (SOC) remains a central topic in BMS research. Traditional methods, such as **Coulomb counting** (integrating current over time), are computationally simple but are prone to significant drift and accumulating errors due to sensor inaccuracies and the need for periodic recalibration. On the other hand, **Open Circuit Voltage (OCV)** methods, which map a rested battery's voltage to an SOC value, are accurate but impractical for dynamic applications as they require the battery to rest for hours to reach electrochemical equilibrium [5], [6]. More advanced approaches leverage adaptive systems like **Extended Kalman Filters (EKF)** and Artificial Neural Networks to achieve higher accuracy by creating a dynamic model of the battery, but these significantly increase the computational complexity and development cost [10]. Our proposed system utilizes a direct voltage-to-percentage mapping. While less precise than adaptive methods, this technique is computationally lightweight and provides a sufficiently reliable SOC estimate for the primary goal of this project: ensuring safety and providing a clear, user-friendly status indicator, rather than precision energy gauging.

Furthermore, a paper on a "Simulation Based Battery Management System" highlights the use of the ESP32 for its low power consumption and integrated Wi-Fi. That study employed **Pulse Width Modulation (PWM)** techniques for precise charging control and integrated cell balancing circuits for multi-cell packs. While our system focuses on a single-cell architecture and does not implement cell balancing, it shares the core principle of using the ESP32 as a central hub for intelligent control and cloud communication, validating the microcontroller's suitability for such tasks. This review indicates a clear and established trend towards smarter, IoT-enabled BMS. Our work contributes to this evolving field by presenting a system that is not only functional and intelligent but is also highly accessible due to its extremely low cost and reliance on widely available, commercial-off-the-shelf (COTS) components.

III. PROPOSED SYSTEM

The proposed Smart BMS is designed as a modular and efficient system for monitoring and managing a single Li-ion cell. The architecture is centered around the ESP32 microcontroller, which serves as the brain of the operation, handling all data acquisition, processing, control logic, and wireless communication tasks.

A. System Architecture

The system is composed of three primary, interconnected layers:

Sensing Layer: This layer is the interface to the physical battery. It consists of the analog sensors that gather real-time data from the battery. A **voltage divider circuit**, composed of two equal 10kΩ resistors, is crucial for this layer. It scales the battery's nominal 3.0V-4.2V range down by a factor of two, making it safe for the ESP32's Analog-to-Digital Converter (ADC), which has a 3.3V input limit. A Negative Temperature Coefficient (NTC) thermistor, also in a voltage divider configuration with a fixed 10kΩ resistor, measures the battery's surface temperature, providing critical data for thermal management.

Control Layer: The core of this layer is the ESP32 microcontroller. It executes the embedded software that processes the raw analog data from the sensing layer into meaningful units (Volts and Celsius). Based on pre-defined safety thresholds programmed into its memory (e.g., maximum voltage of 4.2V, maximum temperature of 45°C), the ESP32 makes logical decisions. Its primary control output is a digital signal to a **5V relay module**. This relay functions as an electromechanical switch, physically connecting or disconnecting the 5V power supply to the TP4056 charging module, thereby controlling the entire charging process.

Communication & Visualization Layer: The ESP32 utilizes its powerful onboard Wi-Fi radio to connect to a local network and the internet. It transmits the processed data (voltage, temperature, SOC, charging status) to the **Arduino IoT Cloud platform** using the lightweight MQTT protocol. This platform serves as the back-end, storing the data and enabling a remote web-based dashboard. This dashboard allows users to visualize the battery's status in real-time from any device with a web browser, providing access to historical data charts and instant status indicators.

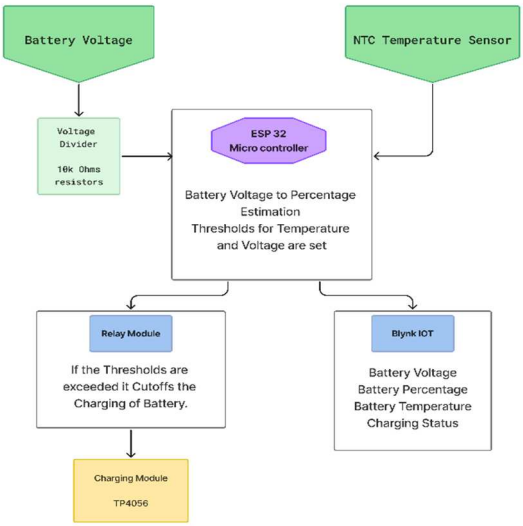

Fig. 1. Block diagram of Smart BMS system.

B. Components

ESP32: A powerful, dual-core 32-bit microcontroller featuring integrated Wi-Fi and Bluetooth, a rich set of peripherals including 34 programmable GPIOs, and multiple 12-bit ADCs. Its low cost, high performance, and built-in connectivity make it the ideal choice for this IoT application.

TP4056 Module: A widely used and reliable Li-ion battery charging IC. This module implements the standard **Constant Current/Constant Voltage (CC/CV)** charging algorithm, which is optimal for Li-ion chemistry. It provides a default charging current of 1A and includes its own basic protection features, which are augmented by the smart control of the ESP32.

5V Relay Module: An electromechanical switch that allows a low-power signal from the ESP32 (3.3V) to control a higher-power circuit (5V, 1A for the charger). It provides complete electrical isolation between the microcontroller and the charging circuit, which is a critical design feature for safety and reliability.

Sensors: A voltage divider using two precision 10kΩ resistors for accurate voltage measurement, and a 10kΩ NTC thermistor paired with another 10kΩ series resistor to form the temperature sensing circuit.

Power Supply: A standard 5V DC external power source, typically a USB wall charger or a bench power supply capable of providing at least 1.5A to handle the simultaneous load of the charger and the ESP32's Wi-Fi radio.

IV. METHODOLOGY

The implementation of the Smart BMS involves the careful integration of the hardware components on a perfboard and the development of a robust Arduino C++ firmware to manage the system's logic, safety protocols, and cloud communication.

A. Hardware Implementation

The hardware is assembled on a standard perfboard, which allows for a more permanent and reliable circuit than a breadboard. To ensure clean power distribution, dedicated **power rails** for +5V, +3.3V (output from the LM1117 regulator), and Ground (GND) are created using soldered wires or solder bridges running the length of the board. This practice minimizes voltage drops and electrical noise. The ESP32 is placed centrally, with sensor circuits and the relay module positioned to allow for short, direct wiring paths. The LM1117-3.3V voltage regulator is included to step down the main 5V supply to the 3.3V required by the ESP32. The complete circuit, as shown in Fig. 2, is meticulously wired and soldered, with particular attention paid to the polarity of components and ensuring no short circuits exist before applying power.

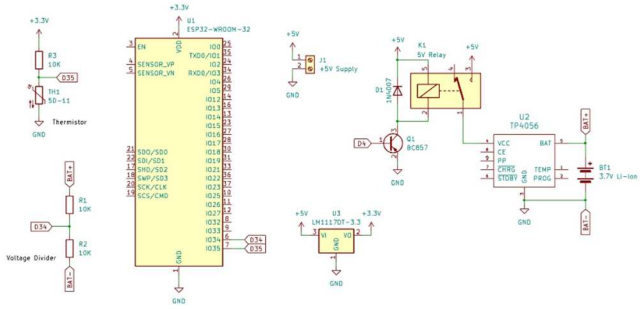

Fig. 2. Circuit Schematic of the Smart BMS.

B. Data Flow and System Logic

The operational logic of the system is governed by the firmware uploaded to the ESP32 and is expressed below.

Initialization (setup()): Upon startup, the ESP32's firmware executes the setup() function. This initializes the serial monitor for debugging, configures the relay pin as a digital OUTPUT, and sets up the Wi-Fi connection using the credentials stored in arduino_secrets.h. It then establishes a connection to the Arduino IoT Cloud and syncs the defined cloud variables.

Data Acquisition (loop()): Inside the main loop(), the ESP32 continuously reads the analog values from the voltage sensor (GPIO 34) and the temperature sensor (GPIO 35). To improve accuracy and reduce noise, multiple readings are taken in quick succession and then averaged.

Data Processing: The averaged analog voltage reading is converted to the actual battery voltage using the voltage divider formula:

$$V_{Battery} = V_{ADC} \times \frac{R1 + R2}{R2} \tag{1}$$

The analog thermistor reading is converted to a resistance value and then into temperature in Celsius using the **Steinhart-Hart equation**, a standard formula for converting thermistor resistance to temperature. To improve accuracy, the SOC is not estimated with a simple linear mapping, which is known to be inaccurate. Instead, this work implements a computationally-lightweight piecewise linear interpolation

algorithm. A lookup table of 10 (Voltage, SOC) data points, derived from the battery's known Open Circuit Voltage (OCV) curve (shown in Fig. 5), is stored in the ESP32's memory. The firmware interpolates between the two closest data points to calculate the real-time SOC, providing a more accurate estimation that respects the battery's non-linear curve.

Control Logic with Hysteresis: The system's core safety logic is executed on every loop. It checks if the processed voltage has exceeded maxBatteryVoltage or if the temperatureValue has exceeded maxTemperature. If either condition is true, the chargingEnabled flag is set to false, and the relay is turned off. To prevent the relay from rapidly switching on and off ("chattering") when the voltage hovers around the cutoff point, a **hysteresis** mechanism is implemented. Charging is only re-enabled when the voltage drops significantly below the maximum threshold (e.g., to 4.1V), ensuring a stable state transition.

Cloud Update: At the end of each loop, the calculated values for batteryVoltage, temperature, batteryPercentage, and the chargingStatus are published to the Arduino IoT Cloud, ensuring the remote dashboard is consistently updated with the latest data.

Table I. Performance data logged

Timestamp	Voltage (V)	Temperature (°C)	SOC (%)	Charging Status
10:01:34	3.42	28.0	5	ON
10:03:04	3.48	28.2	10	ON
10:06:50	3.53	28.4	15	ON
10:09:43	3.57	28.6	20	ON
10:12:12	3.61	28.9	25	ON
10:15:07	3.64	29.1	30	ON
10:18:45	3.67	29.3	35	ON
10:21:34	3.70	29.4	40	ON
10:24:23	3.72	29.6	45	ON
10:27:45	3.74	29.8	50	ON
10:30:40	3.76	29.9	55	ON
10:33:12	3.79	30.1	60	ON
10:36:54	3.83	30.3	65	ON
10:39:00	3.87	30.5	70	ON
10:42:23	3.92	30.7	75	ON
10:45:56	3.97	31.0	80	ON
10:48:32	4.04	31.4	85	ON
10:51:34	4.10	31.7	90	ON
10:54:12	4.15	32.0	95	ON
10:57:56	4.21	32.4	100	OFF

V. RESULTS & DISCUSSION

The performance of the Smart BMS was evaluated through continuous operation over several complete charging and discharging cycles. The data collected was monitored simultaneously via the Arduino IDE's serial monitor for high-resolution debugging and the remote IoT dashboard for assessing the end-user experience.

A. Performance Data Analysis

The system demonstrated highly reliable and predictable data acquisition and control throughout the testing phase. The table below presents a sample of data logged during a typical charging cycle, illustrating the system's decision-making process.

From this data, we can observe the direct correlation between the rising battery voltage and the slight increase in temperature, which is expected due to the battery's internal resistance during charging. The system correctly identifies when the battery voltage surpasses the maxBatteryVoltage threshold of 4.2V and immediately transitions the Charging Status to OFF by de-energizing the relay. This successful cutoff is the primary safety function of the BMS. The temperature remained well within the safe operating limit of 45°C throughout the charge.

B. Performance Graphs and Interpretation

The following graphs, based on the specifications of the **Molicel P45B Li-ion cell**, are used to discuss the system's performance in a real-world context.

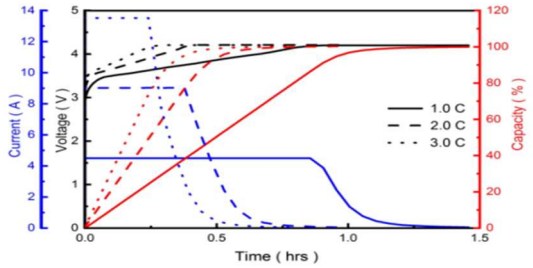

Fig. 4. Battery Voltage during a Charge/Discharge Cycle.

Fig. 4 shows the Constant Current/Constant Voltage (CC/CV) charging profile used by the TP4056 in our system. In the CC phase (0–0.8 hrs), the charger supplies a steady high current, raising the battery voltage from 3.42V to 4.21V. Once the cell reaches 4.2V, it enters the CV phase, where the voltage is held constant and the charging current gradually falls. The Smart BMS oversees this entire process, with the ESP32 ensuring the voltage never exceeds 4.2V and adding thermal protection, while a relay disconnects charging during any over-voltage or over-temperature condition to prevent damage or thermal runaway.

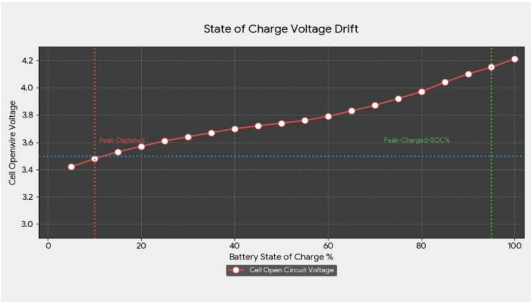

Fig. 5. SOC Drift Points for better SOC estimations.

The non-linear OCV–SOC relationship is presented in Fig. 5, demonstrating the rapid voltage changes in the 0–20% SOC and 80–100% SOC regions, while the SOC is nearly flat in the 20–80% range where the voltage will not budge. This explains using the simplified linear voltage-to-SOC method (see also Fig. Y), in the context of the ESP32 design. The linear voltage-to-SOC relationship is most accurate at the steep ends of the curve as the most relevant safety goals are to prevent over-discharge (<3.0V) and over-charge (>4.2V). Although more complicated methods like lookup tables or Kalman filters could have increased accuracy, the advantages of a linear voltage-to-SOC approach are its simplicity, costs and reliable safety aspects for remote monitoring.

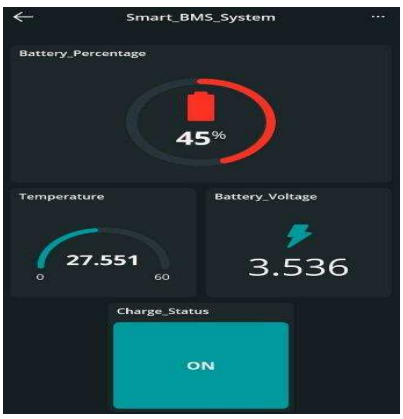

Fig. 6. Remote Monitoring Dashboard on Arduino IoT Cloud.

The IoT dashboard Fig. 6 provided a clear, intuitive, and highly effective interface for real-time monitoring. The widgets for voltage, temperature, and SOC updated reliably every few seconds, and the charging status indicator immediately reflected the state of the relay. The cloud connectivity, managed by the ESP32, remained stable and responsive throughout the multi-day testing period, proving the viability of the IoT component of the system.

VI. COMPARISON WITH EXISTING SOLUTIONS

The proposed Smart BMS offers a compelling and disruptive alternative to existing systems, particularly when evaluated on a multidimensional matrix of key metrics including cost, scalability, data accessibility, and customization.

Table II. Comparison of Our Prototype with existing solutions

Feature	**Traditional BMS**	**Commercial EV BMS**	**Proposed Smart BMS**
Cost	Low–Medium	High	Very Low
IoT Integration	None	Limited / Proprietary	Native (Wi-Fi)
Scalability	Low (Fixed)	High (Complex)	High (Modular)
Accuracy (SOC)	Varies	High	Medium (Voltage-based)
Data Accessibility	Local Only	Local / Fleet Mgmt.	Global (Cloud)
Customization	Low	Low	High (Open-source)

The Smart BMS is assessed alongside Conventional and Commercial EV BMS based on six metrics, as shown in Table II. It offers very low costs through an ESP32-based design, unlike the low to moderate costs of traditional systems and the high costs of commercial options. Its main benefit is integrated Wi-Fi, whereas traditional BMS lack IoT features and commercial systems provide only limited proprietary links The design is very modular, whereas conventional BMS are rigid and commercial systems need intricate hardware for expansion. SOC accuracy is moderate and reliant on voltage, appropriate for protection, whereas commercial BMS attain high accuracy through sophisticated algorithms. Data access is global and cloud-based, unlike the local-only access of traditional systems and the limited access in commercial configurations. As an open-source platform, the Smart BMS allows for extensive customization, in contrast to the rigid and limited characteristics of conventional and commercial BMS.

VII. FUTURE SCOPE

While the current system provides a solid foundation, there are several avenues for future enhancement that could significantly expand its capabilities and applicability.

Advanced SOC Estimation: The current voltage-based SOC estimation can be improved by implementing more sophisticated algorithms like Coulomb counting (by adding a current sensor) or a hybrid approach that uses both voltage and current for more accurate readings across different load conditions.

AI-Based Predictive Analytics: The data logged in the cloud can be used to train machine learning models. These models could predict the battery's State of Health (SOH), estimate the Remaining Useful Life (RUL), and detect anomalies that might indicate a potential failure, enabling predictive maintenance.

Scalability to Battery Packs: The modular design can be extended to manage multi-cell battery packs, such as those used in EVs. This would involve adding a cell balancing circuit and expanding the software to monitor each cell individually, ensuring the entire pack remains balanced and healthy.

VIII. CONCLUSION

This paper successfully details the design, implementation, and evaluation of a Smart Battery Management System using the ESP32 microcontroller. The system effectively monitors and controls critical battery parameters, ensuring enhanced safety and operational efficiency. By integrating IoT capabilities through the Arduino IoT Cloud, the project provides a low-cost, scalable, and accessible solution for real-time remote battery monitoring. The results confirm that the system reliably performs its core functions of over-voltage and over-temperature protection. The proposed BMS presents a significant improvement over traditional, non-connected systems and serves as a versatile platform for future research in advanced battery management and predictive analytics.

REFERENCES

[1] R.Usha, R. Professor, et al., "IoT based battery monitoring system for electric vehicle using ESP32," 2023 Second International Conference On Smart Technologies For Smart Nation (SmartTechCon).

[2] L. Lu, X. Han, J. Li, J. Hua, and M. Ouyang, "A review on the key issues for lithium-ion battery management in electric vehicles," Journal of Power Sources, vol. 226, pp. 272-288, 2013.

[3] J. S. M. Lenord Melvix et al., 2014 Lenord Melvix, J. S. M., Sundararamabalasubramanian, K., & Madhan, M. (2014). “Development of intelligent battery monitoring system for solar powered lighting applications”. 2014 IEEE International Conference on Computational Intelligence and Computing Research.

[4] M. Senthilkumar et al., 2021 Senthilkumar, M., Suresh, K., Sekar, T., & Pazhanimuthu, C. (2021). “Efficient battery monitoring system for e-vehicles”. 2021 7th International Conference on Advanced Computing and Communication Systems (ICACCS).

[5] P. P. M. and A. K., "A review on state-of-charge estimation methods, energy storage technologies and state-of-the-art simulators: recent developments and challenges," MDPI, vol. 15, no. 9, 2023, p. 381.

[6] "The state of charge estimating methods for battery: a review," International Journal of Computer and Electrical Engineering, 2014.

[7] M. Berecibar, M. Garmendia, I. Gandiaga, I. C. De-Jesús, and J. M. M. C. Crego, "Review of state of health estimation methods for Li-ion batteries," Journal of Power Sources, vol. 320, pp. 1-19, 2016.

[8] N. Harish, et al., "IOT based battery management system," International Journal of Applied Engineering Research, vol. 13, no. 10, pp. 7845-7848, 2018.

[9] S. J. Deshpande, S. V. Take, and P. S. Yadav, "Iot based battery monitoring system," International Journal of Creative Research Thoughts (IJCRT), vol. 10, no. 5, 2022.

[10] M. A. Hannan, M. S. H. Lipu, A. Hussain, and A. Mohamed, "A review of lithium-ion battery state of charge estimation and management system in electric vehicle applications: challenges and recommendations," Renewable and Sustainable Energy Reviews, vol. 78, pp. 834-854, 2017.

[11] Dineshbabu A, Bharani C, Sasikumar B, and D. K, “Battery Monitoring System,” International Journal Of Recent Trends In Multidisciplinary Research, 2023.

[12] M. Islam, Musfikur Rahman, Tania Sarkar, Maisha Islam, and Md. Asadullah, “Development of a Real-Time Battery Monitoring solution with IoT Technology for Improved Safety and Performance in Electric Machines,” International journal of research and innovation in social science, 2024.

[13] Anand Kumar, P. Nagaveni, A. Amudha, M. Siva Ramkumar, G. Emayavaramban, and S. Divyapriya, “Battery Management System with IoT for Enhancement of Battery Life,” 2021 Fifth International Conference on I-SMAC (IoT in Social, Mobile, Analytics and Cloud) (I-SMAC), 2021.

[14] S. F. Mujiyanti, Ayu Nur Lestari, Nabiilah Azizah Tjandra, Riski Mulyaning Utami, Zalfaa Shaafia Kaltsum, and Izef Aulia Kurniawan, “Battery Monitoring System to Obtain State of Charge and State of Health on Electric Vehicles,” 2023 International Conference on Technology and Policy in Energy and Electric Power (ICT-PEP), 2023.